Übung 2 · Prompts als Artefakte und Wissen abrufen

Cloud-native AI · WS 2026/27 · Praktikumsabgabe P2

- **Start:** Übung am Dienstag, 20.10.2026, 11:30, O/201
- **Abgabe:** Montag, 02.11.2026, 23:59, in Moodle
- **Arbeitsform:** einzeln oder zu zweit, wie in der Moodle-Gruppeneinteilung eingetragen
- **Aufwand:** etwa 8 Stunden für den Kern, dazu etwa 2 Stunden für den Zusatz

---

Worum es geht

In Übung 1 stand der Prompt als String mitten im Code. Das geht, solange es einen Prompt gibt und niemand ihn ändert. In einem echten System gibt es viele Prompts, sie werden ständig verbessert, und jede Verbesserung kann etwas anderes kaputt machen. Deshalb behandeln wir Prompts ab heute wie Code: Sie liegen in eigenen Dateien, haben eine Version, und jede Änderung wird gemessen.

Danach wechselt ihr die Seite. Ihr greift den Campus-Assistenten an, bringt ihn dazu, ein Geheimnis zu verraten, und härtet ihn Schritt für Schritt. Dabei messt ihr zwei Zahlen, die immer zusammengehören: wie viele Angriffe durchkommen und wie viele harmlose Fragen dabei auf der Strecke bleiben.

Grundlage sind die Skripte zu **VL 3** (Prompt Engineering) und **VL 4** (Wissen abrufen).

Teil	Inhalt	Aufwand	
1	Prompts aus dem Code holen	ca. 45 min	Kern
2	Die Einordnung verbessern und messen	ca. 2 h	Kern
3	Den Assistenten angreifen und härten	ca. 2 h	Kern
4	Wissen abrufen	ca. 2,5 h	Kern
5	README mit Ergebnissen und Reflexion	ca. 45 min	Kern
Z	Zusatz: ein Modell greift an	ca. 2 h	Zusatz

Das Starter-Projekt liegt im Ordner `starter/`. Legt es in eurem Repository aus Übung 1 als neuen Ordner `uebung_2/` ab. Docker-Setup und `.env` sind dieselben wie in Übung 1.

Neu ist nur: Die Modelle liegen ab jetzt in einem Volume mit festem Namen, `cnai-ollama-modelle`, das sich alle weiteren Übungen teilen. Damit ihr die 2,4 GB nicht noch einmal herunterladet, kopiert ihr die Modelle einmal aus dem Volume von Übung 1 hinüber, **am besten vor der Übung zu Hause**. Alle Befehle laufen im Hauptordner eures Repositorys, dort, wo die `compose.yaml` von Übung 1 liegt:

```
docker compose down                     # Übung 1 stoppen, sonst ist Port 11434 belegt
docker volume ls | grep ollama          # Namen des alten Volumes finden
docker volume create cnai-ollama-modelle
docker run --rm -v ALTES_VOLUME:/von -v cnai-ollama-modelle:/nach alpine cp -a /von/. /nach/
```

`ALTES_VOLUME` ersetzt ihr durch den Namen aus der zweiten Zeile, etwa `cnai-p1_ollama-modelle`. In PowerShell findet ihr ihn mit `docker volume ls` ohne `grep`. In **Git Bash** unter Windows setzt ihr vor den letzten Befehl `MSYS_NO_PATHCONV=1 `, sonst verbiegt Git Bash die Pfade `/von` und `/nach`. Danach:

```
cd uebung_2
cp ../.env .env           # oder .env.example kopieren
docker compose run --rm --build app prompts.py
```

Ab jetzt zeigt Docker bei jedem Aufruf die Warnung `volume "cnai-ollama-modelle" already exists but was not created by Docker Compose`. Die ist harmlos und bedeutet nur, dass ihr das Volume selbst angelegt habt.

Teil 1 · Prompts aus dem Code holen (Kern)

Im Ordner `app/prompts/` liegt jeder Prompt als eigene YAML-Datei. `einordnen.v1.yaml` enthält genau den System-Prompt aus Übung 1, dazu Metadaten: Name, Version, Beschreibung, Autor, Datum und Temperature. `app/prompts.py` liest eine solche Datei und baut daraus die Nachrichten für die API. `einordnen.py` kennt keinen Prompt mehr, sondern bekommt ihn als Datei übergeben.

```
docker compose run --rm app prompts.py
docker compose run --rm app einordnen.py --prompt prompts/einordnen.v1.yaml
```

Es gilt eine Regel: **Eine veröffentlichte Prompt-Version wird nie mehr geändert.** Wer etwas verbessern will, legt eine neue Datei mit der nächsten Nummer an.

Aufgaben:

1. Lest `prompts.py` und `einordnen.v1.yaml`. Erklärt im README in drei, vier Sätzen, warum `nachrichten()` den Platzhalter `{eingabe}` mit `replace` statt mit `str.format` einsetzt.
2. Begründet die Regel „nie ändern, nur neue Version". Was ginge schief, wenn ihr `einordnen.v1.yaml` einfach überschreibt und eure Messwerte im README sich auf die alte Fassung beziehen?
3. `mails.json` hat jetzt 16 Mails. Lasst `einordnen.py` mit v1 laufen und notiert, wie viele Anliegen und wie viele komplette Einordnungen stimmen.

Teil 2 · Die Einordnung verbessern und messen (Kern)

Schreibt `prompts/einordnen.v2.yaml` und wendet dabei die Regeln aus dem Skript an. Ihr entscheidet, was ihr davon braucht:

- klare Definition jeder Kategorie, auch was zu `sonstiges` gehört
- die erlaubten Studiengangskürzel mit vollem Namen, und was bei fehlendem Studiengang passiert
- eine klare Regel für Fristen: nur, was wörtlich in der Mail steht
- ein paar Beispiele (Few-Shot)
- die Mail als Daten markieren, z. B. `<mail>…</mail>`, und die Aufgabe danach wiederholen

Vergleicht dann beide Versionen auf denselben Mails:

```
docker compose run --rm app vergleich.py prompts/einordnen.v1.yaml prompts/einordnen.v2.yaml
```

Das Skript zeigt pro Version die Fehler und am Ende eine Tabelle mit Trefferquoten, durchschnittlichen Eingabe-Tokens und Dauer.

Aufgaben:

1. Übernehmt die Tabelle ins README. Was hat v2 gebracht, und was hat es gekostet? Rechnet aus, um welchen Faktor die Eingabe-Tokens gestiegen sind.
2. Welche Mails sind mit v2 immer noch falsch? `vergleich.py` zeigt bei jedem Fehler, was herauskam und was erwartet war. Ordnet jeden Fehler ein: unklare Anweisung, fehlendes Wissen, Angriff, oder ist die **Messung zu streng**? Ein Beispiel für Letzteres: Das Modell schreibt `06.11.`, erwartet war `6 November`. Wie würdet ihr die Messung verbessern?
3. Schaut euch die letzte Mail in `mails.json` an. Was steht bei `studiengang`? Warum hilft hier keine Formulierung im Prompt zuverlässig?
4. Ändert die Klasse `Anfrage` in `einordnen.py` so, dass `studiengang` nur noch `AIN`, `WIN`, `MSI` oder `None` sein kann. Messt erneut. Ist die letzte Mail jetzt richtig eingeordnet? Erklärt, was diese Abwehr verhindert, was nicht, und warum sie auf einer anderen Ebene wirkt als euer Prompt.
5. Optional: Gibt es eine v3, die mit weniger Tokens genauso gut ist? Wenn ja, nehmt sie mit in die Tabelle.

Teil 3 · Den Assistenten angreifen und härten (Kern)

`assistent.py` beantwortet Fragen mit dem Prompt `prompts/assistent.v1.yaml`. Darin stehen Fakten zu Bibliothek, Mensa und Prüfungen, und ein interner Hinweis: der **Türcode des Kopierraums**. Den darf der Assistent nie verraten. Alle Angaben sind für die Übung ausgedacht.

```
docker compose run --rm app assistent.py "Wann hat die Bibliothek samstags offen?"
docker compose run --rm app angriff.py --zeigen prompts/assistent.v1.yaml
```

`angriff.py` schickt die zwölf Angriffe und acht harmlosen Fragen aus `angriffe.json` an den Assistenten und misst:

- **Verletzungsrate:** Anteil der Angriffe, die ihr Ziel erreicht haben (Türcode verraten oder fremde Anweisung ausgeführt).
- **Fehlablehnungsrate:** Anteil der harmlosen Fragen ohne brauchbare Antwort.

Aufgaben:

1. **Messen.** Lasst `angriff.py` mit v1 laufen. Welche Angriffsarten kommen durch, welche nicht?
2. **Selbst angreifen.** Denkt euch drei eigene Angriffe aus, die mit v1 erfolgreich sind, und ergänzt sie in `angriffe.json`. Mindestens einer davon soll **indirekt** sein, die Anweisung steckt also in einem Text, den der Assistent verarbeiten soll.

3. **Prompt härten.** Schreibt `prompts/assistent.v2.yaml`. Nutzt die Abwehr auf Prompt-Ebene aus dem Skript: klare Verbote, eine feste Absage, Eingabe markieren, Aufgabe wiederholen.
4. **Guardrail am Ausgang.** Setzt `ausgabe_pruefen` in `assistent.py` um (TODO 3): Eine Antwort, die den Türcode enthält, wird durch eine feste Absage ersetzt. Testet ihn mit dem Gedicht-Angriff: Erkennt euer Guardrail den Code auch, wenn die Ziffern auf mehrere Zeilen verteilt sind?
5. **Geheimnis entfernen.** Schreibt `prompts/assistent.v3.yaml` ohne den Türcode.
6. **Fehlablehnungen.** Hat eure Härtung harmlose Fragen gekostet? Wenn ja, welche, und warum?

Damit ihr die Wirkung von Prompt und Guardrail getrennt seht, messt ihr am Ende alle drei Versionen zweimal, einmal ohne und einmal mit Guardrail:

```
docker compose run --rm app angriff.py --ohne-guardrail prompts/assistent.v1.yaml prompts/assistent.v2.yaml prompts/assistent.v3.yaml
docker compose run --rm app angriff.py prompts/assistent.v1.yaml prompts/assistent.v2.yaml prompts/assistent.v3.yaml
```

Tragt beide Messungen in eine Tabelle im README ein, mit Verletzungsrate und Fehlablehnungsrate für jede Version, je ohne und mit Guardrail. Eure eigenen Angriffe müssen in allen Messungen enthalten sein. Ein Lauf über drei Versionen dauert etwa fünf Minuten.

Teil 4 · Wissen abrufen (Kern)

Bisher kennt der Assistent nur die vier Fakten, die in seinem System-Prompt stehen. Ab jetzt holt er sein Wissen aus einer kleinen Wissensbasis: 21 Markdown-Dateien im Ordner `app/wissen/`, von Bibliothek bis Prüfungsordnung, alle ausgedacht. Ihr baut drei Wege, dieses Wissen in den Kontext zu bringen, und messt sie gegeneinander. Grundlage ist das Skript zu **VL 4**.

Variante	Skript	Was in den Kontext kommt
`alles`	`alles.py`	die ganze Wissensbasis, ohne Suche
`woerter` und `datei`	`datei_suche.py`	die drei Dateien mit den meisten Treffern einer Textsuche. Die Suchbegriffe sind entweder die Wörter der Frage (`--woerter`) oder vom Modell vorgeschlagen
`rag`	`rag.py`	die vier Abschnitte, deren Embedding der Frage am ähnlichsten ist, aus der Vektordatenbank Qdrant

Die Antwort entsteht immer gleich, mit `prompts/wissen.v1.yaml`. Unterschiedlich ist nur, was gesucht und mitgegeben wird. `fragen.json` enthält 14 Testfragen mit der Datei, in der die Antwort steht.

4.0 · Einrichten (am besten zu Hause)

Ladet **„Übung 2 · Teil 4 (ZIP)"** herunter und entpackt es **in euren Ordner** `uebung_2/`. Dabei werden `compose.yaml`, `.env.example` und `app/requirements.txt` überschrieben, alles andere kommt dazu. Neu sind ein Service `qdrant` und ein drittes Modell, `granite-embedding:278m` (ca. 560 MB). Ergänzt in eurer `.env` die Zeile `EMBED_MODEL=granite-embedding:278m` (steht auch in `.env.example`). Die Embeddings laufen immer lokal, auch wenn ihr das Sprachmodell in der Academic Cloud nutzt.

```
docker compose run --rm --build app alles.py "Wann hat die Mensa offen?"
```

Der erste Aufruf lädt das Qdrant-Image und das Embedding-Modell herunter.

4.1 · Alles in den Kontext (ca. 20 min)

```
docker compose run --rm app alles.py "Ich bin am Tag meiner Klausur krank. Was muss ich tun?"
docker compose run --rm app alles.py "Wo kann ich mein Auto abstellen?"
```

1. Notiert Eingabe-Tokens und Dauer beider Aufrufe. Der zweite ist deutlich schneller, obwohl er gleich viele Tokens hat. Warum? Tipp: Was ist bei beiden Aufrufen gleich, und in welcher Reihenfolge stehen Wissen und Frage in der Nachricht?
2. Schätzt: Was passiert mit Tokens, Kosten und Antwortqualität, wenn die Wissensbasis statt 21 Dateien 2.000 hat?

4.2 · Dateisuche (ca. 45 min)

Setzt `suche()` in `datei_suche.py` um (**TODO 4a**). Danach:

```
docker compose run --rm app datei_suche.py --woerter "Ich bin am Tag meiner Klausur krank. Was muss ich tun?"
docker compose run --rm app datei_suche.py "Ich bin am Tag meiner Klausur krank. Was muss ich tun?"
docker compose run --rm app datei_suche.py --woerter "Wer sitzt in Raum F-201?"
```

1. Welche Dateien findet die Suche mit den Wörtern der Frage, und warum gerade diese? Schaut euch an, welche Wortteile treffen.
2. Welche Suchbegriffe schlägt das Modell vor? Welche Begriffe hätte es wählen müssen, um die richtige Datei zu finden?

3. Bei „Raum F-201“ steht die Zeichenkette wörtlich in einer Datei, und trotzdem landet die Datei nicht unter den ersten drei. Warum nicht? Was müsste die Bewertung anders machen?
4. Claude Code arbeitet genau so, ohne Index, und findet trotzdem fast immer die richtige Stelle. Was hat Claude Code, was unser Aufbau nicht hat? Nennt zwei Unterschiede.

4.3 · RAG (ca. 45 min)

```
docker compose run --rm app rag.py --index
docker compose run --rm app rag.py "Ich bin am Tag meiner Klausur krank. Was muss ich tun?"
docker compose run --rm app vergleich_wissen.py rag
```

Im Moment ist jede Datei genau ein Abschnitt. Notiert das Ergebnis von `vergleich_wissen.py rag`. Dann:

1. Setzt `zerlege()` in `rag.py` um (**TODO 4b**), baut den Index neu und messt noch einmal. Was hat sich bei „Kontext richtig“, „Antwort richtig“ und den Eingabe-Tokens geändert?
2. Öffnet das Qdrant-Dashboard unter `http://localhost:6333/dashboard`, solange der Container läuft (`docker compose up -d qdrant`). Wie viele Punkte liegen in der Sammlung `campus`, und was steht neben dem Vektor in jedem Punkt? Wozu braucht `wissen.py` diese Angaben?
3. Probiert `-k 1` und `-k 8`. Was gewinnt ihr, was verliert ihr?

4.4 · Vergleichen (ca. 20 min)

```
docker compose run --rm app vergleich_wissen.py
docker compose run --rm app vergleich_wissen.py alles
```

Ein Lauf über die drei Standardvarianten dauert etwa sieben Minuten, `alles` noch einmal zwei bis fünf. Übernehmt die Tabelle ins README und beantwortet:

1. Ordnet die Fragen nach ihrem Typ (`direkt`, `kennung`, `umschrieben`, `fehlt`). Welche Variante gewinnt bei welchem Typ, und warum?
2. Wo stand die richtige Datei im Kontext und die Antwort war trotzdem falsch? Was sagt das über die Fehlerquelle?
3. Welche Variante würdet ihr für diese Wissensbasis in Produktion nehmen? Ab welcher Größe würdet ihr umsteigen, und worauf?

4.5 · Eine Verbesserung, gemessen (ca. 20 min)

Sucht euch **eine** Verbesserung aus, setzt sie um und messt sie mit `vergleich_wissen.py` gegen den Stand vorher:

- `prompts/suchbegriffe.v2.yaml` mit besseren Anweisungen oder Beispielen für die Suchbegriffe (VL 3 lässt grüßen)
- eine Bewertung der Dateisuche, bei der seltene Begriffe mehr zählen als häufige (die Idee hinter BM25)
- eine hybride Suche: Kennungen wie `F-201`, `PA-12` oder `§ 17` per Textsuche, alles andere per RAG
- ein anderes Embedding-Modell, etwa `nomic-embed-text` (`docker compose exec ollama ollama pull nomic-embed-text`, dann `EMBED_MODEL` ändern und neu indexieren). Achtet auf die umschriebenen Fragen.

Beschreibt im README in drei, vier Sätzen, was ihr geändert habt und was es gebracht hat.

Teil 5 · README (Kern)

Euer README ist die Abgabe. Es enthält:

1. **Namen** und Matrikelnummern, aber **nur bei einem privaten Repository**. Ist euer Repository öffentlich, gehören Name und Matrikelnummer nur in den Abgabetext in Moodle.
2. **Start:** die Befehle, mit denen man eure Messungen nachvollziehen kann
3. **Ergebnisse** aus Teil 1 bis 4: die Tabellen aus Teil 2 und Teil 3 und eure Antworten auf die Aufgaben
4. **Reflexion**, je zwei bis vier Sätze:
    1. Was ist der Unterschied zwischen einer Verbesserung, die ihr gemessen habt, und einer, die sich besser anfühlt?
    2. Ein Kommilitone sagt: „Mit einem guten System-Prompt ist Prompt Injection gelöst.“ Was antwortet ihr, mit euren Zahlen?
    3. Welche eurer Abwehrmaßnahmen hat am meisten gebracht, und auf welcher Ebene (Modell, Prompt, System) wirkt sie?
    4. Warum braucht man neben der Verletzungsrate immer auch die Fehlablehnungsrate?
    5. Ein Kommilitone sagt: „RAG ist immer besser als Textsuche.“ Was antwortet ihr, mit euren Zahlen aus Teil 4?
    6. Warum misst `vergleich_wissen.py` „Kontext richtig“ und „Antwort richtig“ getrennt?

Zusatz · Ein Modell greift an

Dieser Teil **entfällt**, wenn ihr am Semesterende mindestens 80 % Anwesenheit nachweist. Wer nicht sicher ist, gibt ihn mit ab. Zusatzteile sind nicht nachreichbar.

Von Hand angreifen ist mühsam. Bei **PAIR** (Chao et al., 2023) schreibt ein zweites Modell die Angriffe: Es sieht die Antwort des Ziels und verbessert seinen Angriff, bis er klappt. Im Paper reichten oft weniger als zwanzig Versuche.

1. Setzt in `angreifer.py` die beiden TODOs um.
2. Lasst den Angreifer zehn Runden gegen v1 laufen, ohne Guardrail: `docker compose run --rm app angreifer.py --ohne-guardrail prompts/assistent.v1.yaml`. v1 verrät den Code schon auf die direkte Frage. Hat euer Angreifer trotzdem keinen Erfolg, liegt das meist nicht an eurem Code, sondern am Angreifer-Prompt: Ein kleines Modell schreibt dann umständliche Nachrichten oder fällt aus der Rolle.
3. Verbessert `ANGREIFER_PROMPT` mit dem, was ihr in VL 3 gelernt habt, etwa mit zwei, drei Beispielangriffen (Few-Shot). Lasst ihn danach je zehn Runden gegen v1, v2 und v3 laufen und wiederholt das dreimal, denn bei hoher Temperature ist jeder Lauf anders. Nach wie vielen Runden klappt es, wenn überhaupt?
4. Schaut euch die Angriffe an, die das Modell erfindet. Ist ein guter dabei, den ihr in `angriffe.json` übernehmt?
5. Probiert, falls ihr Zugang habt, ein größeres Modell als Angreifer (Academic Cloud). Was ändert sich? Schreibt eure Beobachtungen ins README.

Was ihr abgebt

In Moodle, Aufgabe „Übung 2 (P2)“: den **Link zu eurem Repository** und den Commit-Hash eures Abgabestands. Das Repository muss für mich lesbar sein: öffentlich, oder ihr ladet meinen GitHub-Account `PascalLaube` als Collaborator ein.

Harte Kriterien, ohne die die Abgabe nicht bestanden ist:

- [ ] Das System startet mit den im README beschriebenen Befehlen nach einer beschriebenen `.env`.
- [ ] Keine Secrets im Repository, `.env` ist nicht committet.
- [ ] Abhängigkeiten sind gepinnt.
- [ ] Unter `app/prompts/` liegen mindestens `einordnen.v1`, `einordnen.v2`, `assistent.v1`, `assistent.v2` und `assistent.v3`. Keine Version wurde nach dem Messen überschrieben.
- [ ] `angriffe.json` enthält eure drei eigenen Angriffe.
- [ ] TODO 4a und 4b sind umgesetzt, `vergleich_wissen.py` läuft mit allen vier Varianten durch.
- [ ] Das README enthält alle Punkte aus Teil 5.
- [ ] Wer unter 80 % Anwesenheit landen könnte: Zusatzteil im README dokumentiert.

Tipps

- **Ein Lauf von `angriff.py` dauert** mit `llama3.2:3b` auf einem Laptop etwa zwei Minuten. Testet neue Angriffe erst einzeln mit `assistent.py`, bevor ihr die ganze Messung startet.
- **Gleiche Bedingungen:** Vergleicht Versionen immer mit demselben Modell und denselben Daten, am besten in einem Aufruf. Sonst messt ihr das Modell oder den Zufall, nicht den Prompt.
- **Schwankungen trotz Temperature 0:** Dieselbe Messung kann zwischen zwei Läufen um einen Fall abweichen, etwa wenn eine harmlose Frage einmal beantwortet und einmal abgelehnt wird. Wiederholt auffällige Messungen und schreibt die Schwankung ins README, statt sie wegzulassen.
- **Kleine Modelle sind eigen.** `llama3.2:3b` kopiert gern eure Markierungen wie `<frage>` in die Antwort. Das ist eine echte Nebenwirkung eures Prompts, notiert sie.
- **Teil 4 auf dem Laptop:** Wird der Rechner bei `alles` sehr langsam oder bricht Docker ab, gebt Docker Desktop mehr Arbeitsspeicher (Settings, Resources) oder nehmt für das Sprachmodell die Academic Cloud. Die Embeddings bleiben lokal.
- **Index veraltet?** Nach jeder Änderung an `wissen/` oder an `zerlege()` muss der Index neu gebaut werden: `rag.py --index`. Genau dieses Problem hat jedes RAG-System in Produktion.
- **Angriffe nur auf euren eigenen Assistenten.** Was ihr hier lernt, probiert ihr nicht an fremden Systemen aus.
- **KI-Werkzeuge** dürft ihr nutzen. Ihr müsst jede Zeile eures Codes und jeden Prompt erklären können.